

CS8165.001 Narrative Reader

A readable original-language companion for Interactive Computer Graphics. Use it with the complete source export and lecture chunks.

Stage	Question to ask
Input	What geometry, attributes, camera, material, and state are provided?
Transform	Which coordinate space is used now?
Clip/Cull	What is removed before rasterization?
Rasterize	Which fragments are generated and which attributes are interpolated?
Shade/Test	Which color/depth result reaches the framebuffer?

01 - Introduction

Source chunk: course_text_parts/03_lectures/01_introduction.txt

Narrative explanation

Computer graphics is the process of turning a model, a camera, light, material, and sampling decisions into pixels. The important exam habit is to avoid treating the final image as one magic step. Every visible result comes from a chain of coordinate changes, geometric decisions, sampling decisions, and shading decisions.

The course starts by separating objects, scenes, cameras, displays, and algorithms. A scene contains geometric data and attributes. A camera defines what part of the world is observed. The rendering algorithm decides how visible surfaces are converted into samples and colors.

For exam preparation, this chapter is the map. Later chapters fill in the transformations, projection, clipping, rasterization, visibility, illumination, texturing, and shadows that make the map operational.

Must know

- Explain graphics as a pipeline from scene data to image samples.
- Separate model data, camera setup, rendering algorithm, and output device.
- Use precise terms: vertex, primitive, fragment, pixel, shader, buffer, sample.

Common pitfalls

- Do not use pixel and fragment as synonyms without context.
- Do not explain an image only from the final color; account for visibility and sampling.

Try it

Draw a one-page map from object data to final image. Add one possible failure at each stage.

Closed checks

1. The output of rendering is best described as:

- A. A set of final image samples
- B. Only a list of triangles
- C. Only a camera matrix

Answer: A

2. A fragment appears before final pixel update.

- A. True
- B. False

Answer: A

02 - Rendering Pipeline

Source chunk: course_text_parts/03_lectures/02_rendering-pipeline.txt

Narrative explanation

The rendering pipeline organizes rendering into predictable stages. Application code supplies data and state. Vertex processing transforms vertices. Primitive assembly creates geometric primitives. Clipping removes parts outside the view. Rasterization creates fragments. Fragment processing computes candidate colors and depths. Per-fragment operations decide what reaches the framebuffer.

A good mental model is that the pipeline keeps changing the representation: vertices become primitives, primitives become fragments, fragments become possible pixel updates. The pipeline is also a debugging map: if geometry is gone, ask whether it failed transformation, clipping, culling, rasterization, or depth testing.

OpenGL exposes this pipeline through buffers, vertex attributes, shader programs, uniforms, textures, and framebuffer state. The exam can ask conceptually or through a small code/debugging scenario.

Must know

- Order the pipeline stages and name their inputs and outputs.
- Distinguish programmable stages from fixed-function decisions.
- Relate OpenGL objects to the stage where they matter.

Common pitfalls

- Do not blame the fragment shader for geometry that never rasterized.
- Do not forget state: depth test, face culling, viewport, and bound objects matter.

Try it

Given a black screen, list five pipeline checkpoints in the order you would inspect them.

Closed checks

1. Rasterization primarily converts:

- A. Primitives into fragments
- B. Textures into vertices
- C. Pixels into triangles

Answer: A

2. Depth testing happens before primitive assembly.

- A. True
- B. False

Answer: B

03 - Geometric Transformations

Source chunk: course_text_parts/03_lectures/03_geometric-transformations.txt

Narrative explanation

Transformations move geometry between coordinate systems. Object coordinates describe a model locally. World coordinates place the model in a scene. View coordinates describe the scene relative to the camera. Clip coordinates prepare geometry for projection and clipping.

Homogeneous coordinates make translation, rotation, scaling, and projection fit into matrix multiplication. The order of multiplication matters because matrix multiplication is not commutative. A rotation followed by a translation is not the same as the reverse.

For exam tasks, write the coordinate space before and after each matrix. This prevents most transformation mistakes and makes formula questions easier to check.

Must know

- Explain homogeneous coordinates and why they are useful.
- Apply translation, rotation, scaling, and composition conceptually.
- Track object, world, view, clip, normalized device, and screen coordinates.

Common pitfalls

- Do not swap transformation order casually.
- Do not mix vectors from different coordinate systems in lighting or camera formulas.

Try it

Create a small chain: model matrix, view matrix, projection matrix. Label the coordinate space after each multiplication.

Closed checks

1. Matrix order matters because:

- A. Matrix multiplication is generally not commutative
- B. Vectors have no coordinates
- C. Projection removes all matrices

Answer: A

2. Translation is naturally represented in a 4x4 homogeneous matrix.

- A. True
- B. False

Answer: A

04 - Geometric Projection

Source chunk: course_text_parts/03_lectures/04_geometric-projection.txt

Narrative explanation

Projection maps view-space geometry into a form that can become a 2D image. Orthographic projection keeps parallel lines parallel and does not shrink distant objects. Perspective projection models foreshortening: objects farther away appear smaller.

The perspective divide is central. After projection, coordinates are divided by the homogeneous component to reach normalized device coordinates. This is why homogeneous coordinates are not just notation; they make perspective projection possible inside the pipeline.

Projection also defines the viewing volume. Near and far planes, field of view, and aspect ratio affect what is visible and how depth precision is distributed.

Must know

- Compare orthographic and perspective projection.
- Explain the role of the perspective divide.
- Relate field of view, aspect ratio, near plane, and far plane to the viewing volume.

Common pitfalls

- Do not describe perspective as only a 2D scaling trick.
- Do not place the near plane at zero in a standard perspective setup.

Try it

Sketch the same cube under orthographic and perspective projection and label the difference.

Closed checks

1. Perspective projection usually makes distant objects:

- A. Appear smaller
- B. Keep exactly the same apparent size
- C. Disappear before clipping

Answer: A

2. The perspective divide happens after projection to reach normalized device coordinates.

- A. True
- B. False

Answer: A

05 - Clipping

Source chunk: course_text_parts/03_lectures/05_clipping.txt

Narrative explanation

Clipping removes parts of primitives outside the view volume. It is not the same as culling and not the same as depth testing. Clipping can create new vertices where a primitive crosses a clipping boundary.

The reason clipping exists is practical: the rasterizer should receive geometry that can be meaningfully converted into fragments inside the view. Clipping also protects the pipeline from invalid geometry around projection boundaries.

In exam answers, always state the clipping space or boundary being used. A vague statement like 'outside the screen is removed' loses precision because clipping normally happens before final screen mapping.

Must know

- Define clipping as cutting primitives against a volume or boundary.
- Distinguish clipping from culling and depth testing.
- Explain why clipping can introduce new vertices.

Common pitfalls

- Do not say clipping removes whole triangles only; partial primitives can be clipped.
- Do not confuse the view volume with the final framebuffer rectangle.

Try it

Draw a line crossing a rectangular window. Mark the kept segment and the two intersection points.

Closed checks

1. When a triangle crosses the view boundary, clipping may:

- A. Create new boundary vertices
- B. Always delete the whole triangle
- C. Only change the texture image

Answer: A

2. Clipping and depth testing solve the same problem.

- A. True
- B. False

Answer: B

06 - Rasterization

Source chunk: course_text_parts/03_lectures/06_rasterization.txt

Narrative explanation

Rasterization determines which samples or pixels are covered by a primitive and creates fragments for later processing. It bridges continuous geometry and the discrete image grid.

This chapter is where sampling, interpolation, and aliasing become visible. Attributes such as depth, color, normals, and texture coordinates can be interpolated across a primitive before fragment shading.

Exam questions often test the difference between the geometric primitive and the generated fragments. A triangle may cover many samples, no samples, or a pattern of samples depending on placement and sampling rules.

Must know

- Explain rasterization as primitive-to-fragment conversion.
- Describe attribute interpolation across a primitive.
- Connect sampling decisions to aliasing and image quality.

Common pitfalls

- Do not say rasterization computes final visibility by itself.
- Do not forget that interpolation happens before many fragment computations.

Try it

Place a small triangle over a pixel grid and mark which samples become fragments.

Closed checks

1. Rasterization produces:

- A. Fragments
- B. Only final visible pixels
- C. Only vertex shader code

Answer: A

2. Texture coordinates can be interpolated across a primitive.

- A. True
- B. False

Answer: A

07 - Visibility Determination

Source chunk: course_text_parts/03_lectures/07_visibility-determination.txt

Narrative explanation

Visibility determination decides what can be seen from the current viewpoint. Back-face culling can remove geometry based on orientation. View-frustum culling can reject objects outside the camera volume. Depth buffering resolves occlusion per fragment by comparing depth values.

The Z-buffer is a central practical algorithm: keep a depth value for each sample or pixel and update the color only when a fragment is closer according to the chosen depth test.

Visibility is view-dependent. The same object can be visible, hidden, clipped, or culled depending on the camera, projection, and state.

Must know

- Explain culling, clipping, and depth testing separately.
- Describe the Z-buffer idea and its update rule.
- Name depth precision issues and why near/far settings matter.

Common pitfalls

- Do not call culling a pixel-level decision.
- Do not ignore depth buffer initialization and depth test state in OpenGL debugging.

Try it

Simulate two overlapping fragments with depths 0.2 and 0.7 under a less-than depth test.

Closed checks

1. The Z-buffer stores:

- A. Depth values used for visibility
- B. Only texture colors
- C. Shader source code

Answer: A

2. Back-face culling is based on orientation, not per-pixel depth.

- A. True
- B. False

Answer: A

08 - Local Illumination

Source chunk: course_text_parts/03_lectures/08_local-illumination.txt

Narrative explanation

Local illumination estimates the color of a surface point from local information such as normal direction, light direction, view direction, material coefficients, and light intensity. It does not fully simulate indirect global light transport unless an approximation is added.

Typical components are ambient, diffuse, and specular terms. Diffuse reflection depends on the angle between the normal and the light direction. Specular reflection models shiny highlights and also depends on the view direction.

For exam work, normalize vectors and state the coordinate system. Many wrong lighting answers come from mixing spaces or using an unnormalized normal.

Must know

- Separate ambient, diffuse, and specular contributions.
- Explain the role of normals, light direction, view direction, and material parameters.
- Compare Phong-style and Blinn-Phong-style specular reasoning qualitatively.

Common pitfalls

- Do not use a normal from object space with a light vector in view space.
- Do not call local illumination a full solution for indirect light.

Try it

For a surface facing away from a light, decide what happens to the diffuse term.

Closed checks

1. Diffuse reflection mainly depends on:

- A. The angle between normal and light direction
- B. Only the screen resolution
- C. Only the texture filename

Answer: A

2. Local illumination normally ignores full indirect light transport.

- A. True
- B. False

Answer: A

09 - Texturing

Source chunk: course_text_parts/03_lectures/09_texturing.txt

Narrative explanation

A texture is sampled data used during rendering. It is often an image, but conceptually it can store many kinds of values. Texture coordinates map a point on a primitive to a lookup in texture space.

Filtering decides how a texture value is reconstructed from discrete texels. Nearest filtering is simple but blocky. Linear filtering blends neighboring texels. Mipmapping helps reduce aliasing when a texture is seen at smaller scale.

Texturing is connected to rasterization because interpolated texture coordinates are used per fragment. It is connected to shading because the sampled value can affect color, normals, material coefficients, or other shader inputs.

Must know

- Define texture coordinates and texture sampling.
- Compare nearest filtering, linear filtering, and mipmapping qualitatively.
- Name different uses of textures beyond color maps.

Common pitfalls

- Do not define a texture as only a picture pasted onto an object.
- Do not confuse geometry resolution with texture sampling quality.

Try it

Explain why a distant checkerboard texture can shimmer without mipmapping.

Closed checks

1. Mipmaps mainly help with:

- A. Reducing aliasing for minified textures
- B. Replacing the projection matrix
- C. Deleting hidden triangles

Answer: A

2. Texture coordinates are interpolated during rasterization for fragment use.

- A. True
- B. False

Answer: A

10 - Shadows

Source chunk: course_text_parts/03_lectures/10_shadows.txt

Narrative explanation

Shadows are another visibility problem: is a surface point visible from the light source? Shadow mapping answers this with two passes. First render depth from the light's point of view into a shadow map. Then render the camera view and compare each relevant point against the stored light-space depth.

The method is practical and widely used, but it has artifacts. Shadow acne comes from self-shadowing precision issues. Peter panning can appear when bias is too large. Resolution and projection choices affect shadow sharpness and stability.

The exam pattern is predictable: explain the two passes, identify what the shadow map stores, describe the comparison, and name common artifacts plus their causes.

Must know

- Explain shadows as light-source visibility.
- Describe the two-pass shadow mapping algorithm.
- Name shadow acne, bias, peter panning, resolution limits, and sampling issues.

Common pitfalls

- Do not say a shadow map stores final colors; it stores depth from the light.
- Do not treat bias as free: too little and too much both cause artifacts.

Try it

Write the two shadow-mapping passes as pseudocode with one line for the depth comparison.

Closed checks

1. A shadow map stores:

- A. Depth from the light's view
- B. Final camera colors
- C. Only mesh filenames

Answer: A

2. Bias can reduce acne but may detach shadows.

- A. True
- B. False

Answer: A

Software And OpenGL Try-Out Labs

Black Screen Debugging

Task: You render a triangle but the window is black. Choose the first checks in pipeline order.

Expected reasoning: Check shader compilation/linking, vertex array/buffer binding and attribute layout, transformation/projection, viewport, draw call, face culling, depth test, and fragment output.

Depth Buffer Experiment

Task: Render two overlapping triangles with and without depth testing. Predict the difference before running it.

Expected reasoning: Without depth testing, later draw order can dominate. With depth testing, the nearer fragment wins according to the depth function and depth buffer contents.

Texture Filtering Toggle

Task: Switch between nearest, linear, and mipmapped filtering on a checkerboard texture.

Expected reasoning: Nearest looks blocky, linear blends neighboring texels, and mipmapping reduces shimmer/aliasing when the texture is minified.

Shadow Bias Slider

Task: Increase and decrease shadow bias in a shadow-mapping example.

Expected reasoning: Too little bias causes self-shadowing acne. Too much bias can detach the shadow from the caster.